- Memento (save/restore an object's state without revealing implementation details)
- Observer (mechanism for multiple objects to receive notifications about events)
- State (allows an object to change behavior based on internal state)
- Template Method (partially define an algorithm in a superclass; subclasses can override arbitrary steps)
- Visitor (traverse an object structure with a set of algorithms held in a special class)

We'll end our quick journey into design patterns here. The scope of this section is after all to simply introduce the topic and encourage further reading. Hopefully you'll also have spotted that there are strong elements of the SOLID principles to be found in design patterns. For example, the Strategy Pattern demonstrates dependency inversion (by using an interface for the `Strategy` classes), and single responsibility (by moving each strategy into a dedicated class). The Visitor Pattern, although not shown in detail here, is a good example of the open-closed principle: by placing new behavior in the special visitor object, we ensure that the visited object structure remains closed to modification.

As for further reading, there's a wealth of material available online, and a certain PHP-specific text (edited by yours truly) has been mentioned multiple times already. I encourage you to explore!

Summary

Object-orientation by itself is not going to ensure your code is amazing. Like any set of programming language features, it needs to be used correctly. In Chapter 6, "Object Fundamentals," we learned how PHP makes objects work. This chapter was more concerned with how to actually use them well. For that we need to think beyond programming and dip our toes into the world of analysis and design.

Hopefully, by giving you a basic introduction to the concept of the problem and solution spaces and modular decomposition, I've whetted your appetites for more study. I promise this will pay dividends if you intend to pursue programming as (at the very least) a meaningful pastime. The more thought you give to why your software needs to exist, who is going to use it, and for what, the better your code will be. In particular, a